

Problem Statement: Student Subject-wise Marks Analysis Using Jagged Arrays

Develop a Java application to manage and analyze the marks obtained by students in different subjects using a class, objects, and a jagged array.

A university wants to maintain the marks of students where **each student may be enrolled in a different number of subjects**. Therefore, a regular two-dimensional array is not suitable because every student may have a different number of subjects. Use a **jagged array** to store the marks.

Create a class named `Student` with the following attributes:

- `studentId`
- `studentName`
- `int[][] marks`

The `marks` jagged array should store marks for multiple students, where each row represents one student and the number of elements in each row represents the number of subjects taken by that student.

Requirements

Create appropriate constructors and methods in the `Student` class to:

1. Accept student details.
2. Create and initialize a jagged array based on the number of subjects taken by each student.
3. Accept marks for each subject.
4. Display the marks of each student.
5. Calculate and display the **total marks** obtained by each student.
6. Calculate and display the **average marks** of each student.
7. Identify the student with the **highest total marks**.
8. Display the number of subjects taken by each student.

Create a separate `StudentDemo` class containing the `main()` method.

The `main()` method should:

- Create one or more `Student` objects.
- Accept the required information from the user.
- Invoke the appropriate methods using the objects.
- Display the complete student performance report.

Java classes provide the blueprint for state and behavior, while objects are created from those classes and used to invoke instance methods. ([Oracle Docs](#))

Example Jagged Array Structure

For three students:

Student 1 → 5 subjects → {78, 85, 67, 90, 88}
Student 2 → 3 subjects → {92, 81, 76}
Student 3 → 4 subjects → {69, 74, 88, 91}

The underlying structure could be represented as:

```
marks[0] → 78  85  67  90  88
marks[1] → 92  81  76
marks[2] → 69  74  88  91
```

This demonstrates that the rows of a Java multidimensional array can have different lengths.
([Oracle Docs](#))

Sample Output

===== STUDENT PERFORMANCE REPORT =====

Student ID : 101
Student Name : Rahul

Subjects : 5
Marks : 78 85 67 90 88
Total Marks : 408
Average : 81.60

Student ID : 102
Student Name : Priya

Subjects : 3
Marks : 92 81 76
Total Marks : 249
Average : 83.00

Student ID : 103
Student Name : Aman

Subjects : 4
Marks : 69 74 88 91
Total Marks : 322
Average : 80.50

=====

Top Performer:
Student Name : Priya

Total Marks : 249

Concepts to be Demonstrated

This single problem can assess:

- **Class and Object**
- Instance variables
- Constructors
- Instance methods
- `this` keyword
- Object creation using `new`
- One-dimensional arrays
- **Jagged arrays**
- Nested `for` loops
- Array traversal
- Passing arrays to methods
- Returning values from methods
- Encapsulation
- User input using `Scanner`

Recommended Extension

For a slightly more advanced version, ask students to create a **student object for each student** and store those objects in an array:

```
Student[] students = new Student[numberOfStudents];
```

Each `Student` object should maintain its own jagged marks array. This makes the problem a stronger demonstration of the relationship between **objects and arrays**, rather than merely using a jagged array inside a class. Java arrays can themselves contain reference types, including class objects. ([Oracle Docs](#))

Suggested title for the lab:

“Student Performance Management System Using Classes, Objects and Jagged Arrays”